LexAI: The Complete Backend Learning Guide & Viva Preparation

Welcome to your complete LexAI Backend Master Guide. This document contains everything you need to understand your project from absolute zero, run it, debug it, and defend it during your MCA Viva.

MODULE 1: Project Structure and Execution Flow

1. The Folder Structure

- `app.py` : **The Boss.** Starts the Flask server and glues everything together.
- `config.py` : **The Rulebook.** Stores environment variables (DB URLs, API keys).
- `extensions.py` : **The Tools.** Sets up your database (`db`) and login manager.
- `models/database_models.py` : **The Blueprints.** Defines MySQL tables (Users, Documents).
- `routes/` : **The Receptionists.** Catches browser requests (e.g., `/login` , `/upload`).
- `services/` : **The Workers.** Does the heavy lifting (Extracting PDF text, ChromaDB, Gemini).

2. Architecture Diagram (How it flows)

```
Browser -> app.py -> routes/ -> services/ -> ChromaDB & Ge
```

3. Deep Dive: `app.py`

This is the entry point of your application.

Line / Code	Simple Meaning	Why It Is Needed
`from flask import Flask`	Imports the web framework.	Like grabbing your toolbox before building.
`from extensions import db`	Imports your shared database tool.	So all files share one DB connection.
`def create_app():`	Function that builds the app.	Allows safe configuration (App Factory pattern).
`app.register_blueprint(auth_bp)`	Attaches your separate route files.	Keeps `app.py` clean instead of writing 1,000 routes here.

Line / Code	Simple Meaning	Why It Is Needed
`app.run(port=5000)`	Turns the server on.	Opens the "door" so browsers can connect.

Explain Like I'm 5: `app.py` is the General Manager of a hotel. It turns on the lights (`Flask`), reads the rulebook (`config.py`), hires the receptionists (`register_blueprint`), and unlocks the front doors (`app.run`).

MODULE 2: Python & Flask Fundamentals

- **What is a Backend?** The engine of the car. The user only sees the steering wheel (frontend), but the backend makes it move (databases, AI).
- **What is Flask?** A lightweight Python tool to build web servers.
- **What is a Route?** A specific URL path (like `/dashboard`). If the website is a house, routes are the different rooms.
- **GET vs POST:** GET is asking for data (like opening a webpage). POST is sending data (like submitting a login form).

MODULE 3: Routes, Requests, and Responses

Let's look at how a route works. In `routes/document_routes.py` , you have an upload route.

Line / Code	Simple Meaning	Why It Is Needed
`@document_bp.route('/upload', methods=['POST'])`	Listens for POST requests at `/upload`.	Tells Flask where to send uploaded files.
`@login_required`	Security check.	Only logged-in users can upload files.
`file = request.files['document']`	Grabs the uploaded file.	Gets the physical file the user sent.
`db.session.add(new_doc)` `db.session.commit()`	Saves record to MySQL.	Permanently saves the file metadata.
`return jsonify({"status": "success"})`	Sends JSON response back.	Tells the frontend Javascript the upload worked.

Explain Like I'm 5: A Route is like a receptionist desk. The `@route` is the sign on the desk ("Uploads Here"). `@login_required` is the bouncer checking ID. `request.files` is taking the package from the user, and `return jsonify` is handing them a receipt.

MODULE 4: Database Models & SQLAlchemy

Your database (`models/database_models.py`) uses SQLAlchemy (an ORM). An ORM lets you write Python code instead of raw SQL queries.

The Tables

1. **User:** Stores `id`, `name`, `email`, `password_hash`.
2. **Document:** Stores `id`, `user_id` (Foreign Key), `file_path`. Belongs to a User.
3. **DocumentAnalysis:** Stores `summary`, `risks` in JSON format.
4. **Conversation & Message:** Stores chat history.
5. **KnowledgeBase:** Stores admin-curated legal concepts.

Explain Like I'm 5: A database table is just a massive Excel spreadsheet. The ORM is a translator. Instead of writing SQL (`SELECT * FROM users`), you write Python (`User.query.all()`), and the ORM translates it for MySQL.

MODULE 5: Authentication & Security

Located in `routes/auth_routes.py`. * **Password Hashing:** You use `Flask-Bcrypt`. If a user's password is "password123", Bcrypt scrambles it into "$2b$12$xyz...". Even if a hacker steals the database, they cannot read the passwords. * **Flask-Login:** Remembers who is logged in using secure browser cookies.

MODULE 6 & 7: Processing & Vector Databases (ChromaDB)

When a PDF is uploaded: 1. **Extraction:** `PyMuPDF` reads the file and extracts raw text. 2. **Chunking:** LLMs cannot read 500-page books at once. We slice the text into 500-word "chunks" with overlapping sentences so context isn't lost. 3. **Embeddings:** `SentenceTransformers`

`(all-MiniLM-L6-v2)` converts each chunk of text into a massive array of numbers (a vector). 4. **ChromaDB:** Stores these vectors. We attach `user_id` metadata to every vector so User A can never accidentally search User B's contract.

Explain Like I'm 5: Embeddings turn concepts into GPS coordinates. "Dog" and "Puppy" have similar GPS coordinates. ChromaDB is just Google Maps for text.

MODULE 8: Gemini Integration & RAG

RAG (Retrieval-Augmented Generation) is the heart of LexAI. 1. User asks: "What is the penalty?" 2. **Retrieval:** ChromaDB finds the chunks of the contract mathematically closest to the word "penalty". 3. **Augmentation:** We inject these chunks into a strict prompt. 4. **Generation:** Gemini reads the prompt and answers.

Why RAG? If you ask Gemini directly, it might hallucinate (invent fake laws). RAG forces Gemini to read *only* your specific contract.

MODULE 9: Error Handling & Fallback

In your chat services, you use `try... except`. If the Gemini API crashes (e.g., no internet, API limit), the `except` block catches the error. Instead of crashing the whole website, your Fallback parser simply retrieves the raw contract text from ChromaDB and shows it to the user.

MODULE 10: Running on Windows & Troubleshooting

How to Run LexAI

1. Open PowerShell in your `LexAI` folder.
2. Activate environment: `venv\Scripts\activate`
3. Install dependencies: `pip install -r requirements.txt`
4. Ensure MySQL is running (e.g., MySQL96 service).
5. Start app: `python app.py`
6. Open browser: `http://127.0.0.1:5000`

Troubleshooting

Error	Cause	Fix
`ModuleNotFoundError`	Forgot to activate venv.	Run `venv\Scripts\activate`.
`Access denied for user`	Wrong MySQL password in `.env`.	Open `.env` and fix `SQLALCHEMY_DATABASE_URI`.
`Port 5000 is in use`	Another server is running.	Restart PC or change port in `app.py`.

---

40 MCA VIVA QUESTIONS & ANSWERS

1. **Why did you choose Flask over Django?** Flask is a micro-framework, perfect for building AI-heavy microservices without unnecessary bloat.
2. **What is an ORM?** Object-Relational Mapping. It lets us interact with MySQL using Python classes instead of writing raw SQL queries.
3. **How does LexAI connect to MySQL?** Via `Flask-SQLAlchemy`. The connection string is loaded from `config.py` into

`db.init_app(app)`.

4. **How are passwords stored?** We use Bcrypt to hash and salt passwords before storing them in MySQL.
5. **How do you prevent a user from reading another user's contract?** In ChromaDB, every text chunk is saved with metadata: `{"user_id": current_user.id}`. Every search strictly filters by this ID.
6. **What library extracts PDF text?** PyMuPDF (fitz).
7. **What library extracts DOCX text?** python-docx.
8. **Why do we 'chunk' text?** Because LLMs like Gemini have strict token limits and lose context if you feed them 100 pages at once.
9. **What is an Embedding?** A mathematical vector representation of text. Similar meanings produce vectors that are close together in multidimensional space.
10. **Which embedding model did you use?** SentenceTransformers `all-MiniLM-L6-v2`. It is fast and runs locally.
11. **Why ChromaDB?** It is an open-source, local vector database optimized for storing and retrieving embeddings quickly.
12. **What is RAG?** Retrieval-Augmented Generation. We *retrieve* facts from ChromaDB, *augment* the prompt, and *generate* an answer with Gemini.
13. **Does RAG stop hallucinations completely?** No, but it drastically reduces them by grounding the LLM's responses in verified retrieved text.
14. **How does the Fallback mechanism work?** If the Gemini API times out, a `try/except` block catches the error and returns the raw retrieved text from ChromaDB instead.
15. **What is a Blueprint in Flask?** A way to organize routes. Instead of 100 routes in `app.py`, we split them into `auth_routes`, `chat_routes`, etc.
16. **How does `@login_required` work?** It is a Python decorator from Flask-Login that checks for an active session cookie before allowing access to a route.
17. **What is semantic search?** Searching by meaning. Keyword search looks for "car"; semantic search understands that "automobile" is the same thing.

18. **What does `request.files` do?** It catches physical files (like PDFs) uploaded from an HTML form via a POST request.
19. **What does `jsonify()` do?** It converts Python dictionaries into JSON format so the frontend JavaScript can read it easily.
20. **What is DOMPurify used for?** It sanitizes Markdown to HTML on the frontend to prevent Cross-Site Scripting (XSS) attacks.
21. **How is the Knowledge Base different from Document Chat?** The Knowledge Base uses a completely separate ChromaDB collection curated only by admins for general legal questions.
22. **What is `__name__` in Flask?** It is a Python variable that tells Flask where the current file is located so it can find the `templates` and `static` folders.
23. **What happens if I upload an .exe file?** The backend validates the file extension against an `ALLOWED_EXTENSIONS` set. If it's an `.exe`, the backend rejects it.
24. **How do you handle Database Migrations?** (If asked): Using Flask-Migrate or by manually writing scripts to update schemas without losing data.
25. **What is a Primary Key?** A unique identifier for a database record (e.g., `user.id`).
26. **What is a Foreign Key?** A column that links to a Primary Key in another table (e.g., `document.user_id` links to `user.id`).
27. **What does `db.session.commit()` do?** It permanently saves the pending database transaction to MySQL.
28. **What happens if a database operation fails halfway?** We use `db.session.rollback()` inside an `except` block to undo changes and prevent database corruption.
29. **What is Cosine Similarity?** A mathematical metric used by ChromaDB to measure how close two vectors (and therefore two text chunks) are to each other.
30. **Where are the actual PDFs saved?** In the local `uploads/` directory on the server file system. Only the file path is saved in MySQL.
31. **Why not save PDFs directly in MySQL?** Storing large binary blobs in a relational database slows it down significantly. File systems are

built for files.

32. **What is the `config.py` file for?** It centralizes configuration like the `SECRET_KEY`, database URI, and debug settings so they aren't hardcoded everywhere.
33. **What is a Virtual Environment (venv)?** An isolated Python environment. It ensures LexAI's specific library versions don't conflict with other projects on the PC.
34. **How does the frontend communicate with the backend chat route?** Using AJAX/Fetch API. It sends a JSON POST request and waits for a JSON response without reloading the page.
35. **What is the Gemini API?** Google's Generative AI endpoint. We send it a structured prompt containing the user's question and the ChromaDB context, and it returns a formulated answer.
36. **What is a System Prompt?** Instructions hidden from the user that tell the AI how to behave (e.g., "You are a legal assistant. Answer only using the provided text.").
37. **What is the biggest limitation of LexAI currently?** It lacks OCR (Optical Character Recognition), meaning it cannot read scanned physical paper documents, only digital PDFs.
38. **How would you improve this project in the future?** Integrate OCR, add support for regional languages, and migrate the local ChromaDB to a cloud-hosted vector database.
39. **Is LexAI meant to replace lawyers?** No, it is an assistive tool meant to increase efficiency and democratize basic legal understanding, but human verification is always required.
40. **Can you explain the entire flow in 10 seconds?** User uploads PDF -> Flask extracts and chunks text -> SentenceTransformers embeds chunks into ChromaDB -> User asks question -> ChromaDB retrieves relevant chunks -> Gemini generates answer based on chunks -> Flask sends answer to browser.

End of Complete Backend Guide.